

As you can see, for parameters or variables, the type hint uses a colon to separate the name from the type, whereas for return values, the type hint uses an arrow (`->`) to separate the `def` statement's closing parentheses from the type. The `describeNumber()` function's type hints show that it takes an integer value for its `number` parameter and returns a string value.

If you use type hints, you don't have to apply them to every bit of data in your program. Instead, you could use a *gradual typing* approach, which is a compromise between the flexibility of dynamic typing and the safety of static typing in which you include type hints for only certain variables, parameters, and return values. But the more type hinted your program is, the more information the static code analysis tool has to spot potential bugs in your program.

Notice in the preceding example that the names of the specified types match the names of the `int()` and `str()` constructor functions. In Python, *class*, *type*, and *data type* have the same meaning. For any instances made from classes, you should use the class name as the type:

```
import datetime
❶ noon: datetime.time = datetime.time(12, 0, 0)

class CatTail:
    def __init__(self, length: int, color: str) -> None:
        self.length = length
        self.color = color

❷ zophieTail: CatTail = CatTail(29, 'grey')
```

The `noon` variable has the type hint `datetime.time` ❶ because it's a `time` object (which is defined in the `datetime` module). Likewise, the `zophieTail` object has the `CatTail` type hint ❷ because it's an object of the `CatTail` class we created with a `class` statement. Type hints automatically apply to all subclasses of the specified type. For example, a variable with the type hint `dict` could be set to any dictionary value but also to any `collections.OrderedDict` and `collections.defaultdict` values, because these classes are subclasses of `dict`. Chapter 16 covers subclasses in more detail.

Static type-checking tools don't necessarily need type hints for variables. The reason is that static type-checking tools do *type inference*, inferring the